# Linux and the shell

Software Management & Development · Session 2




IE University · MFT & MCSDI

# Installs to start now

They download while we talk.

## macOS

- Terminal → paste the command from brew.sh. It asks for your password and may install Apple's developer tools: let it run
- Meanwhile, download Docker Desktop (Apple Silicon) from docker.com

## Windows (and anyone using a VM)

- Start your Ubuntu or Debian VM and open its terminal: everything happens inside the VM
- `sudo apt update && sudo apt install -y curl git`, then Docker: `curl -fsSL https://get.docker.com | sh`

> Docker needs ≥ 20 GB free and macOS 15 or later. Not your case? Skip Docker today and tell me.

# Why use the shell

How much did we spend on restaurants last quarter, and where?

```
grep ",restaurants," transactions.csv | cut -d, -f3 | sort | uniq -c | sort -rn
```

```
   5 Rodilla
   4 La Tagliatella
   2 Five Guys
```

The command is the record of the analysis: save it, share it, run it next quarter, run it on a server.

> Servers, containers and the cloud are operated from a shell. Often it's the only interface they have.

# Terminal, shell and other words

The terminal is the window; the shell is the program inside it that runs your commands.

| Word | What it is | Examples |
|---|---|---|
| Terminal | the window where you type and read | macOS Terminal, Windows Terminal, VS Code |
| Shell | the program that runs your commands | zsh, bash, PowerShell |
| Command line, CLI | working by typing commands instead of clicking | `git`, `docker`, `uv` |
| Prompt | what the shell shows when it waits for you | `ana@laptop ~ %` |
| Script | a file of commands run one after another | `balance.sh` |

# Small tools, combined

- Each program does one thing well
- Programs work together: the output of one can be the input of the next
- Text is the universal interface

The restaurants pipeline, one program at a time:

```
grep ",restaurants," transactions.csv | cut -d, -f3 | sort | uniq -c | sort -rn
```

| | |
|---|---|
| `grep ",restaurants,"` | keeps the restaurant rows |
| `cut -d, -f3` | keeps the third column: the name |
| `sort` | puts equal names together |
| `uniq -c` | counts each run of equal names |
| `sort -rn` | orders by that count, largest first |

# The shell and the filesystem

# Parts of a command

`ls` `-l -h` `/etc`

command options argument

- Options change the behaviour: `-lh` = `-l -h`; long options use `--all`
- Spaces separate arguments: quote `"my file.csv"`
- Every command returns an exit code: `0` = success. `echo $?` shows the last one

```
ls -lh --all ~/smd          # two short options and a long one
cp "my file.csv" backup/    # two arguments: the quotes keep the first one whole
ls nofile; echo $?          # not 0: the command failed
```

# The filesystem tree

| | |
|---|---|
| `/` | the root |
| `/home/<user>` | your files (macOS: `/Users`) |
| `/etc` | system configuration |
| `/usr/bin`, `/bin` | programs |
| `/usr/local/bin` | programs installed by hand |
| `/var/log` | logs |
| `/tmp` | temporary files |

- Absolute path: from the root, `/home/ana/smd/ledger`. Works from anywhere
- Relative path: from the current directory, `ledger/data`. Depends on where you are
- `~` is your home folder: `~/smd` = `/home/ana/smd`

# Navigating folders

| | |
|---|---|
| `pwd` | where am I |
| `cd ledger` | go to a folder |
| `cd ..` | one level up |
| `cd ~` | back to your home folder |
| `ls -la` | list, with hidden files |
| `./balance.sh` | a file in this folder (`.` = here) |

```
$ pwd
/home/ana
$ cd smd/ledger
$ ls
data
$ cd ..          # back to /home/ana/smd
```

# Working with files

| | |
|---|---|
| `mkdir -p a/b` | create folders |
| `cp`, `mv` | copy, move or rename |
| `rm`, `rm -r` | delete, for good |
| `cat`, `less` | print, page through |
| `head`, `tail` | first and last lines |

```
cp data/transactions.csv data/backup.csv
head -n 3 data/transactions.csv
```

> There is no recycle bin.
>
> Read every `rm` twice.
>
> Never `rm -rf` a path you haven't checked.

# Keyboard shortcuts

| | |
|---|---|
| Tab | complete names |
| ↑ | previous commands |
| ↓ | next commands |
| Ctrl+R | search the history |
| Ctrl+C | stop the command |
| Ctrl+L | clear the screen |
| `man cmd` | the command's manual (q to quit) |
| `cmd --help` | a shorter summary of the options |

> Press Tab often: if it doesn't complete, the name is wrong or the file isn't where you think.

# stdin, stdout and stderr

Every program starts with three channels open, together called standard I/O (stdio). The shell decides where each one goes.

| Stream | Number | By default | Used for |
|---|---|---|---|
| stdin | 0 | the keyboard | what the program reads |
| stdout | 1 | the terminal | its results |
| stderr | 2 | the terminal | errors and warnings |

```
ls data nofile > list.txt     # the error still appears on screen
```

> Errors have their own stream so they don't end up mixed with your data, and you still see them when the results go to a file.

# Redirection and pipes

| | |
|---|---|
| `cmd > f` | stdout to `f`, replacing it |
| `cmd >> f` | append to `f` |
| `cmd < f` | stdin from `f` |
| `cmd 2> f` | stderr (stream 2) to `f` |
| `a \| b` | stdout of `a` → stdin of `b` |

```
grep groceries data/transactions.csv > groceries.csv
echo "Groceries: 12 rows" >> report.txt
wc -l < groceries.csv                  # 12
ls nofile 2> errors.txt
```

# Text-processing tools

| | |
|---|---|
| `grep -i -v -c` | filter lines |
| `cut -d, -f3` | one column |
| `sort -rn` | sort |
| `uniq -c` | count adjacent repeats |
| `wc -l` | count lines |
| `awk -F,` | columns, filters, sums |

```
cut -d, -f4 data/transactions.csv | tail -n +2 | sort | uniq -c | sort -rn | head -n 3
```

```
  12 groceries
  11 restaurants
   9 transport
```

> `sort | uniq -c | sort -rn | head`: “what is most frequent?” for any column of any file.

# Users, packages and the environment

# File permissions

```
$ ls -l ~/smd/ledger
drwxr-xr-x  2 ana  ana   4096 Oct 29 10:02 data
-rwxr-xr-x  1 ana  ana    252 Oct 29 10:05 balance.sh
-rw-r--r--  1 ana  ana    180 Oct 29 10:07 report.txt
drwx------  2 ana  ana   4096 Oct 29 10:08 private
$ ls -l /etc/passwd /etc/shadow
-rw-r--r--  1 root root    888 Oct 24 14:12 /etc/passwd
-rw-r-----  1 root shadow  502 Oct 24 14:12 /etc/shadow
```

Type (`d` folder, `-` file), then owner, group and others. File: r read, w write, x run. Folder: r list, w add or delete inside, x enter.

| Name | Type | Owner | Group | Others |
|---|---|---|---|---|
| `data` | `d` folder | `rwx` everything | `r-x` list, enter | `r-x` list, enter |
| `balance.sh` | `-` file | `rwx` read, write, run | `r-x` read, run | `r-x` read, run |
| `report.txt` | `-` file | `rw-` read, write | `r--` read | `r--` read |
| `private` | `d` folder | `rwx` everything | `---` nothing | `---` nothing |
| `/etc/shadow` | `-` file | `rw-` read, write | `r--` read | `---` nothing |

# Changing permissions and owners

| | |
|---|---|
| `chmod +x balance.sh` | let everybody run it |
| `chmod go-r report.txt` | remove read for group and others (u owner, g group, o others, a all) |
| `chmod -R g+w data` | let the group write in the folder and everything inside it |
| `chmod 644 report.txt` | `rw-r--r--`: the usual for files |
| `chmod 755 balance.sh` | `rwxr-xr-x`: the usual for scripts and folders |
| `chmod 600 key.pem` | `rw-------`: only the owner (keys, passwords) |
| `chmod 700 private` | `rwx------`: only the owner can enter |
| `sudo chown ana report.txt` | make `ana` the owner |
| `sudo chown ana:staff report.txt` | change owner and group |
| `sudo chown -R ana:ana data` | the folder and everything inside it |

Numbers: r = 4, w = 2, x = 1, added per group: 7 = `rwx`, 6 = `rw-`, 5 = `r-x`, 4 = `r--`, 0 = `---`. Giving files to another user takes `sudo`.

# root and sudo

- root can do anything
- `sudo cmd` runs one command as root, with your password, and logs it
- Least privilege: work as a normal user

```
$ apt install tree
E: Could not open lock file /var/lib/dpkg/lock-frontend - open (13: Permission denied)
$ sudo apt install tree
[sudo] password for ana:
```

> `sudo` and `chmod 777` are not fixes. Find out why it failed first.

# Package managers

| System | Tool | Install | Update all |
|---|---|---|---|
| Ubuntu / WSL | `apt` | `sudo apt install tree` | `sudo apt update && sudo apt upgrade` |
| macOS | `brew` | `brew install tree` | `brew update && brew upgrade` |
| Windows | `winget` | `winget install Git.Git` | `winget upgrade --all` |

- Trusted repositories, dependencies resolved, repeatable: a list of packages is a reproducible install
- `curl … | sh` runs code you haven't read: only from official sources

```
$ brew install tree          # or: sudo apt install tree
$ which tree
/opt/homebrew/bin/tree       # Ubuntu: /usr/bin/tree
```

# Environment variables

Named text values that every process receives and passes on to the programs it starts.

| | |
|---|---|
| `HOME` | your home folder: `/home/ana` |
| `USER` | your user name |
| `SHELL` | your shell: `/bin/zsh`, `/bin/bash` |
| `PATH` | where commands are found (next slide) |

```
echo $HOME
LEDGER_FILE=x.csv ./balance.sh  # one command
export LEDGER_FILE=x.csv        # this session
# permanent: add the export to ~/.zshrc
```

> No spaces around `=`. Without `export`, programs don't see it.

# PATH: where commands are found

A list of folders. The shell searches them in order and runs the first match:

```
$ echo $PATH | tr ":" "\n"
/opt/homebrew/bin
/usr/local/bin
/usr/bin
/bin
$ which python3
/opt/homebrew/bin/python3
```

- `command not found` → folder not in `PATH` (or open a new terminal)
- wrong version runs → another one comes first
- `./script.sh`: the current folder isn't in `PATH`

# Setting up the course environment

# Setup options: three tracks

| Track | What you get | Who it suits |
|---|---|---|
| A. Native | your OS's terminal, package manager and Docker | macOS and Linux |
| B. Linux VM | the Ubuntu or Debian VM from the intro course, Docker inside | everybody on Windows; anyone who wants isolation |
| C. Cloud | GitHub Codespaces: Linux + VS Code in the browser | managed or low-spec laptops |

> Same commands on every track: you always work in a Linux-style shell. In the VM, keep projects in `~` inside the VM, not in a shared folder.

# WSL: only if you know what you're doing

The Windows Subsystem for Linux runs Linux programs on Windows, without dual boot and without managing a VM yourself. When it breaks, fixing it takes experience: in this course, use the VM unless you already master WSL.

- WSL 2 is a real Linux kernel in a lightweight VM that Windows starts and stops for you
- On top, a distribution: Ubuntu, with bash, `apt` and the usual tools
- Docker Desktop runs its Linux containers in the same VM

## Integrated with Windows

- Ubuntu is a tab in Windows Terminal
- Linux files appear in File Explorer, under Linux
- The Windows disk is at `/mnt/c` from Linux, but it's slow: work in `~`
- VS Code opens Linux folders

# Checking the installation

```
echo $SHELL
uname -sm
curl --version | head -n 1
brew --version     # macOS
apt --version      # Ubuntu or Debian (VM)
docker run --rm hello-world
```

- `brew: command not found` → run the two “Next steps” commands Homebrew printed
- `Cannot connect to the Docker daemon` → start Docker Desktop (macOS) or `sudo systemctl start docker` (Linux, VM); `permission denied … docker socket` → log out and in
- Windows: work in the VM’s terminal, not in PowerShell; projects in `~` inside the VM